FOREST FIRE DETECTION

Image-Based Forest Fire Detection Using SVM and CNN

Academic Case Study Report

Project Type: Academic Machine Learning and Web Application Project

Technology Stack: Python, Flask, OpenCV, NumPy, scikit-image, scikit-learn, Joblib, TensorFlow and Keras

1. Executive Summary

The Forest Fire Detection project implements an image-based classification system that analyzes a forest image and predicts whether a fire is detected. The supplied implementation compares two machine learning approaches: a classical Support Vector Machine (SVM) using Histogram of Oriented Gradients (HOG) features and a Convolutional Neural Network (CNN) using full-color images.

A Flask web application provides a common interface for uploading an image, selecting SVM, CNN or a comparison mode, and displaying the prediction with a confidence value and safety message. The project demonstrates the complete inference path from image acquisition and preprocessing to feature extraction or neural-network preprocessing, model execution and browser presentation.

2. Problem Statement

Forest fires can spread rapidly and cause ecological, economic and human impacts. Image-based automated detection can assist monitoring systems by identifying visual fire indicators from camera or uploaded image data. A practical academic prototype therefore needs a repeatable image-processing pipeline and a user interface that can present a model prediction clearly.

3. Objectives

- Develop an image-based forest fire detection application.
- Implement a classical SVM pipeline using HOG image features.
- Implement a CNN inference pipeline using a saved TensorFlow model.
- Provide a common Flask interface for both models.
- Allow users to upload an image and obtain a fire/no-fire prediction.
- Display model confidence and a safety-oriented response.
- Provide a comparison mode that executes both models on the same input image.
- Package the trained model artifacts with the application for local inference.

4. Scope of the System

The supplied project focuses on inference rather than model training. The repository contains a serialized SVM model and a TensorFlow SavedModel for CNN inference. The application accepts common image files, preprocesses them according to the selected model and returns a classification. The project is an academic prototype and should not be treated as an operational wildfire warning system without field validation, calibrated probabilities, robust datasets and appropriate safety procedures.

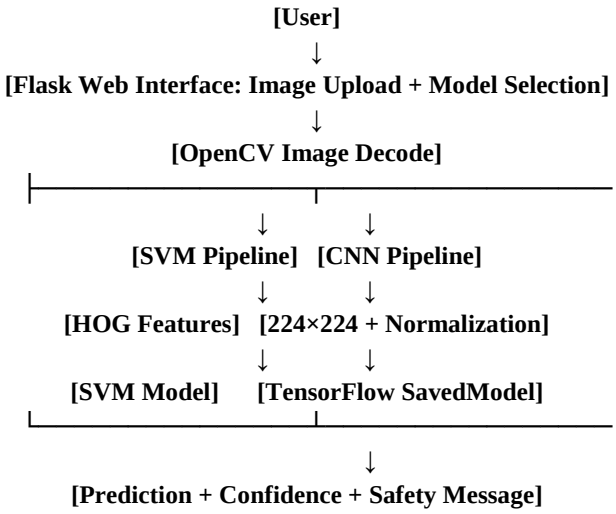
5. Functional Requirements

Module	Implemented Capability
Image Input	Upload a forest image through the Flask interface.
SVM Detection	Resize to 128×128, convert to grayscale, extract HOG features and classify with the trained SVM.
CNN Detection	Resize to 224×224, normalize pixel values to [0,1] and perform

	inference using the SavedModel.
Model Comparison	Run both SVM and CNN on the same uploaded image and display both results.
Confidence Display	Convert the relevant model output into a percentage for presentation.
Safety Message	Present a response indicating whether a possible fire was detected.
Web Interface	Render all results through a Flask/Jinja2 template and CSS.

6. System Architecture

System Flow



7. SVM Methodology

The SVM branch uses a hand-crafted feature representation rather than passing raw color pixels directly to the classifier. Each input image is resized to 128×128 pixels and converted from BGR to grayscale. HOG features are then extracted with 9 orientations, a 16×16 pixel cell size, 2×2 cells per block and L2-Hys block normalization.

The resulting feature vector is reshaped into a single sample and passed to the serialized SVM model loaded through Joblib. The application uses predict() for the class and predict_proba() to obtain the maximum class probability for the displayed confidence value.

8. CNN Methodology

The CNN branch preserves the color information of the input image and resizes it to 224×224 pixels. Pixel values are divided by 255 so that the input values are normalized to the [0,1] interval. The image is expanded with a batch dimension and passed to the TensorFlow SavedModel through Keras TFSMLayer using the serving_default endpoint.

The application reads the returned tensor and compares the resulting probability with a configured threshold of 0.182. This threshold is documented in the supplied project as having been selected through precision-recall analysis. The application labels the image as Fire Detected when the probability is above the threshold and No Fire Detected otherwise.

9. Model Comparison Workflow

1. The user uploads an image and selects comparison mode.
2. OpenCV decodes the uploaded bytes into an image array.
3. The SVM pipeline creates the HOG representation and obtains an SVM class and probability.
4. The CNN pipeline creates the normalized 224×224 input and obtains the neural-network probability.
5. The application converts both outputs into presentation-friendly results.

6. The browser displays the SVM and CNN predictions and their corresponding confidence values.

10. Model and Artifact Inventory

Artifact	Role
model/fire_svm_model.pkl	Serialized SVM classifier loaded with Joblib.
model/saved_model.pb	TensorFlow SavedModel definition used for CNN inference.
model/fingerprint.pb	SavedModel fingerprint metadata.
model/variables/variables.index	TensorFlow model variable index.
model/variables/variables.data-00000-of-00001	TensorFlow model variable data.
app.py	Flask routes, preprocessing, inference and result rendering.
templates/index.html	Web interface template for upload and prediction results.
static/style.css	Frontend presentation rules.

11. Technology Stack

Technology	Purpose
Python	Application and inference programming language.
Flask	Web application framework and request routing.
OpenCV	Image decoding, resizing and grayscale conversion.
NumPy	Numerical array operations and input shaping.
scikit-image	HOG feature extraction.
scikit-learn	SVM model implementation and probability inference.
Joblib	Loading the serialized SVM model.
TensorFlow / Keras	Loading and executing the CNN SavedModel.
HTML / CSS / Jinja2	Browser-based user interface.

12. Implementation Details

The Flask application loads both models once when the application starts. This avoids repeatedly deserializing the SVM and loading the TensorFlow model for every request. The root route accepts GET and POST requests. On POST, the uploaded image is decoded with cv2.imdecode(), after which the selected model determines the preprocessing and inference path.

The SVM feature extraction function performs resizing, grayscale conversion and HOG computation. The CNN preprocessing function performs resizing, floating-point normalization and batch-dimension expansion. The comparison branch executes both pipelines independently and returns their outputs to the same template.

13. User Interface Workflow

1. Open the Flask application in a browser.
2. Select SVM, CNN or comparison mode.
3. Upload a supported image such as JPG or PNG.
4. Submit the image for analysis.
5. Review the classification result and confidence.
6. In comparison mode, inspect both model outputs for the same image.

14. Testing Strategy

Test Category	Test Case	Expected Result
File Input	Upload a valid image file	Image is decoded successfully and passed to the selected pipeline.
SVM Preprocessing	Process an image through the SVM branch	128×128 grayscale image produces HOG features and a prediction.
CNN Preprocessing	Process an image through the CNN branch	224×224 normalized batch is passed to the

		SavedModel.
SVM Prediction	Run the serialized SVM model	Fire/no-fire class and probability are returned.
CNN Prediction	Run the SavedModel	Probability is compared with the configured threshold.
Comparison	Select compare mode	Both model predictions and confidence values are displayed.
Invalid/Unexpected Input	Upload unsupported or unreadable content	Application should be extended with explicit validation and user-friendly error handling.

15. Results and Observations

The implemented system provides a single browser-based interface for two distinct computer-vision approaches. The SVM path demonstrates feature engineering through HOG, while the CNN path demonstrates end-to-end learned visual representation through a saved deep-learning model. The comparison mode makes it possible to observe how the two inference approaches behave on the same image.

The repository structure also separates model artifacts, application code and frontend resources. Loading models at startup improves request-time efficiency compared with loading model files inside each request. The threshold used by the CNN is explicitly represented in the application, making the decision rule visible and configurable.

16. Limitations

- The application performs image-level classification and does not provide geographic localization, fire perimeter estimation or temporal tracking.
- A confidence or probability value should not automatically be interpreted as a calibrated real-world probability of fire.
- The CNN threshold of 0.182 is application-specific and depends on the evaluation process used when the model was developed.
- The supplied requirements.txt does not list TensorFlow and Keras even though app.py imports and uses them for CNN inference; a fresh environment therefore needs those dependencies installed separately.
- The application currently runs Flask with debug=True in the supplied app.py; debug mode should be disabled for production use.
- Operational wildfire detection requires extensive validation across different weather, lighting, camera, vegetation and geographic conditions.

17. Future Enhancements

- Add explicit file validation, size limits and robust exception handling.
- Evaluate models using precision, recall, F1-score, ROC-AUC and precision-recall curves on a held-out test set.
- Calibrate model probabilities and document the selected operating threshold.
- Add video-stream or camera-feed inference for continuous monitoring.
- Add temporal smoothing so isolated false detections do not immediately trigger an alert.
- Integrate geolocation metadata and a map-based monitoring dashboard.
- Add explainability methods such as HOG visualization for SVM or saliency/Grad-CAM-style analysis for CNN outputs.
- Use a production WSGI server and containerized deployment with health checks and structured logging.

18. Conclusion

The Forest Fire Detection project demonstrates a practical machine-learning inference application that combines classical computer vision and deep learning. The SVM pipeline illustrates feature engineering with HOG descriptors, while the CNN pipeline illustrates normalized image inference using a TensorFlow SavedModel. Flask provides a lightweight interface that unifies these approaches and presents their results to the user.

19. References and Project Sources

1. Flask documentation:

https://flask.palletsprojects.com/

2. OpenCV documentation:

https://docs.opencv.org/

3. scikit-image documentation:

https://scikit-image.org/docs/

4. TensorFlow documentation:

https://www.tensorflow.org/

5. Keras documentation:

https://keras.io/